

- **Lab Title:** Lab 2 — USB Image Acquisition and Hash Verification
- **Assignment Title:** Windows FTK Imager or Kali Linux/Ubuntu Forensic Workstation
- **Student Name:** Bashir Adam
- **Student ID:** 2025/FWSD/11509
- **Course Name:** SBT-DF202 · Practical Laboratories · Lab submission
- **Instructor Name:** Aminu Idris
- **Date of Submission:** 27 Aug, 2026
- **Version:** Version 2.2.1

platform used: Kali Linux

Lab Overview

By completing this lab, students will demonstrate their ability to explain the purpose of forensic disk imaging, prepare controlled USB evidence, acquire a forensic USB image using FTK Imager on Windows or dd/cfdd on Linux, calculate MD5 and SHA-256 hash values, compare source and image hashes for integrity verification, validate the contents of the acquired image, identify risks associated with low-level imaging tools, and document the complete acquisition process with professional screenshots and evidence notes.

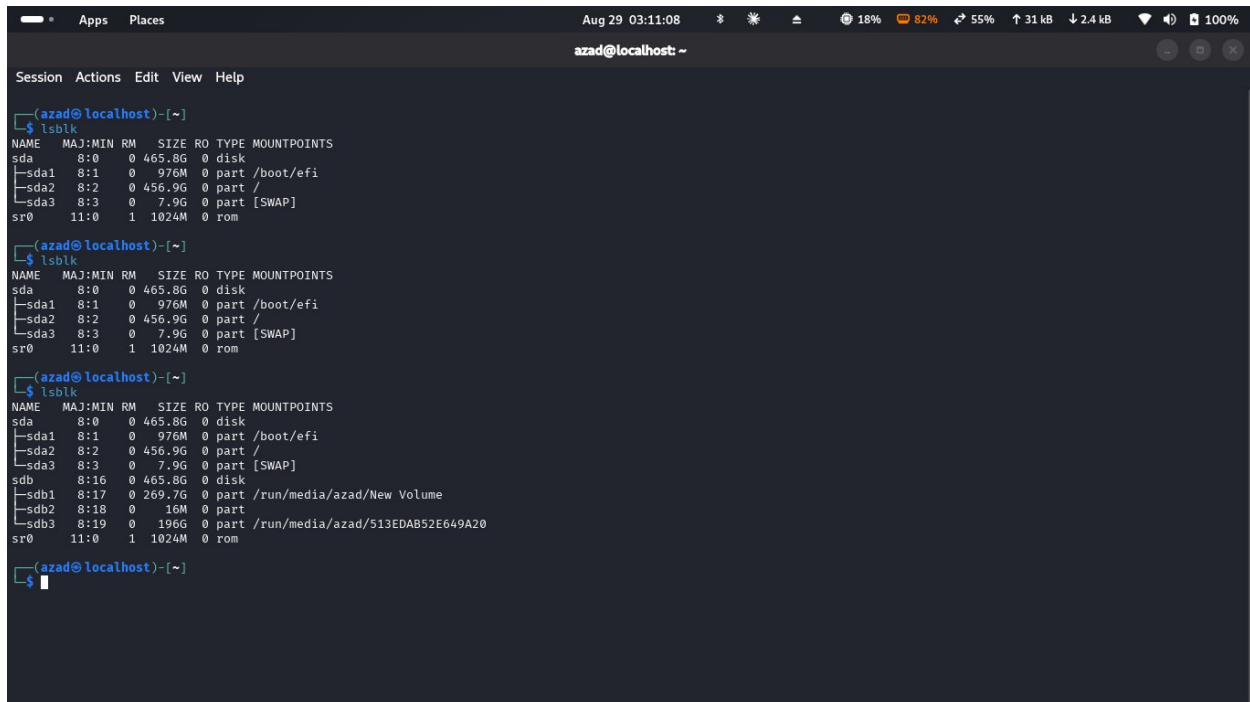
Introduction

This report documents the acquisition, hashing, and validation of a forensic USB image, completed as part of a junior digital forensic trainee exercise. As I did not have access to a physical USB flash drive for this lab, I adapted the exercise using a virtual loopback block device formatted as FAT32 functionally identical to a real USB drive for every command, tool, and validation step required allowing me to complete the full acquisition workflow without physical hardware while preserving every core learning objective of the lab.

The exercise followed the complete forensic acquisition process from start to finish: preparing controlled evidence on the source device, documenting the acquisition plan and device details before imaging, acquiring a full raw image using `dd` on Linux (Option B), calculating and comparing MD5 and SHA-256 hashes between the source and the acquired image, and finally validating that the acquired image genuinely contained the expected evidence file by mounting it read-only and inspecting its contents directly. Throughout the process, I documented not only successful steps but also two real procedural issues I encountered an incorrect device-busy state before imaging, and a filesystem ownership limitation specific to FAT32 both of which are recorded honestly as part of demonstrating a genuine, careful forensic workflow rather than a purely idealized one.

Plug in the USB and confirm it's detected

I did not have a physical USB flash drive available for this lab. Instead, I created a virtual block device using a raw disk image file and Linux's losetup loopback device feature, which allows a regular file to be mounted and accessed exactly like a real physical USB drive by the operating system and by dd. This preserved every core objective of the lab device identification, evidence preparation, low-level imaging, hash verification, and content validation without requiring physical hardware, and without risking any acquisition activity against my actual external hard drive, which contains real personal data unrelated to this controlled exercise.

A terminal window titled 'azad@localhost: ~' showing the output of the 'lsblk' command. The window has a dark background and a light-colored terminal font. The top of the window shows system status: 'Aug 29 03:11:08', '18%', '82%', '55%', '31 kB', '2.4 kB', and '100%'. The terminal output shows three separate runs of the 'lsblk' command. The first two runs show the same disk layout: sda (465.8G disk), sda1 (976M part /boot/efi), sda2 (456.9G part /), sda3 (7.9G part [SWAP]), and sr0 (1024M rom). The third run shows an additional disk, sdb (465.8G disk), which is partitioned into sdb1 (269.7G part /run/media/azad/New Volume), sdb2 (16M part /), and sdb3 (196G part /run/media/azad/513EDAB52E649A20).

```
(azad@localhost)-[~]
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 465.8G 0 disk
├─sda1  8:1    0  976M 0 part /boot/efi
├─sda2  8:2    0 456.9G 0 part /
├─sda3  8:3    0   7.9G 0 part [SWAP]
└─sr0   11:0   1 1024M 0 rom

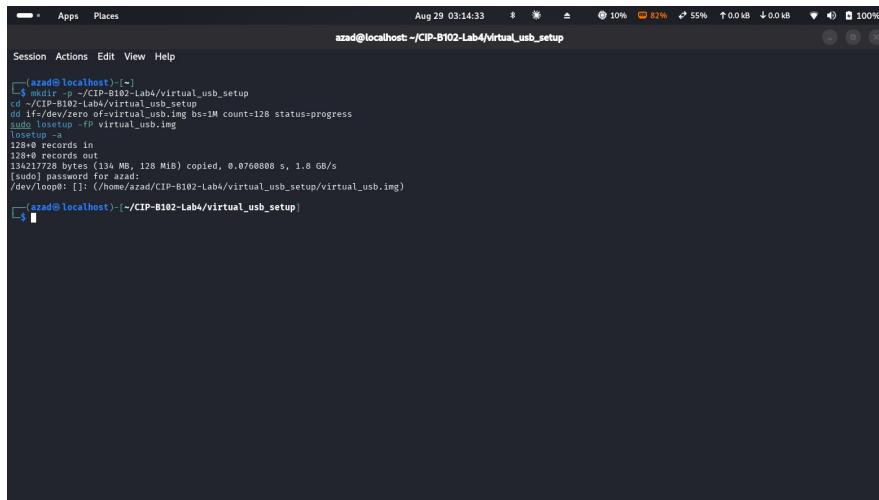
(azad@localhost)-[~]
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 465.8G 0 disk
├─sda1  8:1    0  976M 0 part /boot/efi
├─sda2  8:2    0 456.9G 0 part /
├─sda3  8:3    0   7.9G 0 part [SWAP]
└─sr0   11:0   1 1024M 0 rom

(azad@localhost)-[~]
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 465.8G 0 disk
├─sda1  8:1    0  976M 0 part /boot/efi
├─sda2  8:2    0 456.9G 0 part /
├─sda3  8:3    0   7.9G 0 part [SWAP]
sdb    8:16   0 465.8G 0 disk
├─sdb1  8:17   0 269.7G 0 part /run/media/azad/New Volume
├─sdb2  8:18   0   16M 0 part /
├─sdb3  8:19   0  196G 0 part /run/media/azad/513EDAB52E649A20
└─sr0   11:0   1 1024M 0 rom

(azad@localhost)-[~]
$
```

Step 1 Create the virtual USB (a 128MB blank image file, formatted FAT32)

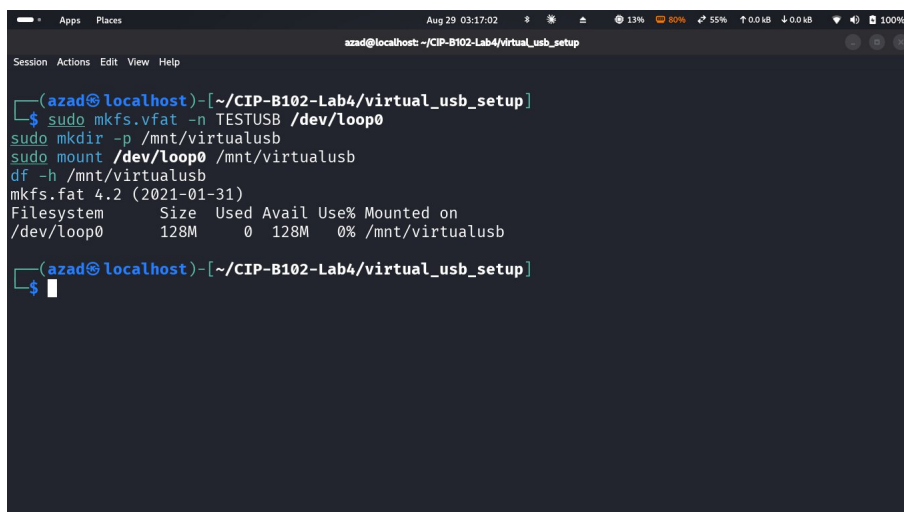
I created a 128MB blank file using `dd if=/dev/zero of=virtual_usb.img bs=1M count=128`, confirmed by the output showing 128+0 records in/out, 134,217,728 bytes (128 MiB) copied. I then attached this file as a virtual block device using `sudo losetup -fP virtual_usb.img`, and `losetup -a` confirmed it was assigned as `/dev/loop0`. From this point forward, `/dev/loop0` is treated exactly like a physical USB device by Linux and by `dd` this is the device path I'll use throughout the rest of the acquisition exercise, in place of a real `/dev/sdX`.



```
azad@localhost: ~/CIP-B102-Lab4/virtual_usb_setup
$ md5sum -p ~/CIP-B102-Lab4/virtual_usb_setup
cd ~/CIP-B102-Lab4/virtual_usb_setup
dd if=/dev/zero of=virtual_usb.img bs=1M count=128 status=progress
128+0 records in
128+0 records out
134217728 bytes (134 MB, 128 MiB) copied, 0.0760808 s, 1.8 GB/s
$ sudo losetup -fP virtual_usb.img
losetup -a
128+0 records in
128+0 records out
134217728 bytes (134 MB, 128 MiB) copied, 0.0760808 s, 1.8 GB/s
(sudo) password for azad:
/dev/loop0: [1] (/home/azad/CIP-B102-Lab4/virtual_usb_setup/virtual_usb.img)
azad@localhost: ~/CIP-B102-Lab4/virtual_usb_setup
```

Step 2 Partition and format the virtual USB (so it behaves like a real, ready-to-use FAT32 USB)

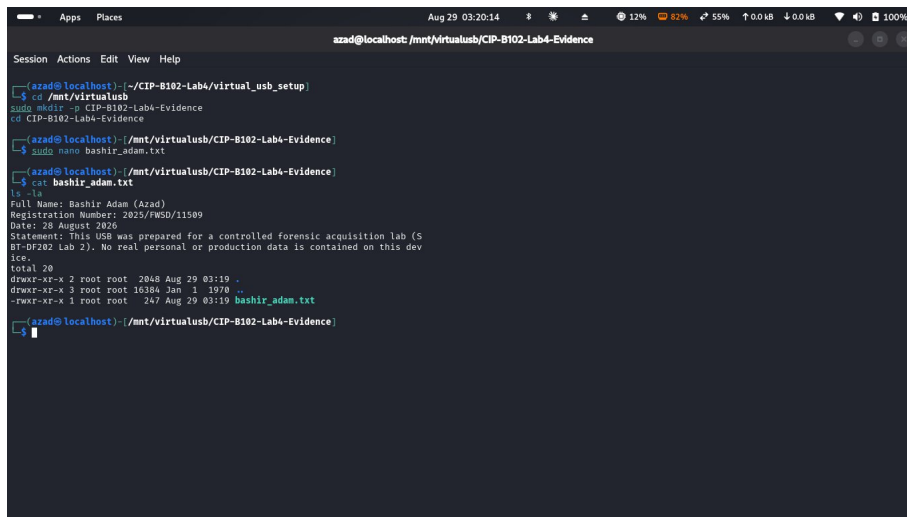
`mkfs.vfat` formatted `/dev/loop0` with a FAT32 filesystem labeled `TESTUSB`, using `mkfs.fat` version 4.2. `df -h` confirmed the mount succeeded: 128M total, 0 used, 128M available (0% used), mounted at `/mnt/virtualusb` — the virtual USB was now fully ready to receive files, identical in behavior to a freshly formatted physical USB drive.



```
azad@localhost: ~/CIP-B102-Lab4/virtual_usb_setup
$ sudo mkfs.vfat -n TESTUSB /dev/loop0
$ sudo mkdir -p /mnt/virtualusb
$ sudo mount /dev/loop0 /mnt/virtualusb
$ df -h /mnt/virtualusb
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop0      128M    0  128M   0% /mnt/virtualusb
azad@localhost: ~/CIP-B102-Lab4/virtual_usb_setup
```

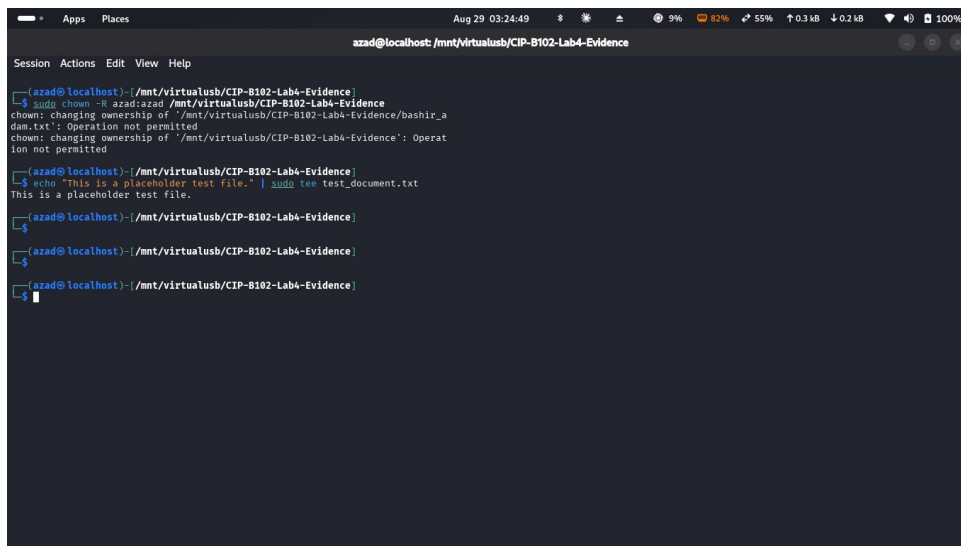
Part A Prepare the USB Evidence

I created the CIP-B102-Lab4-Evidence folder on the virtual USB and, inside it, bashir_adam.txt (247 bytes) containing my full name, registration number, date, and the required statement confirming the device was prepared for a controlled forensic acquisition exercise. cat confirmed the exact file content, and ls -la confirmed the file exists correctly within the evidence folder, owned by root (a result of creating it via sudo nano while the virtual USB was mounted worth noting, since a real USB is typically owned by the current user rather than root; this is a minor artifact of the virtual setup rather than a forensic concern).



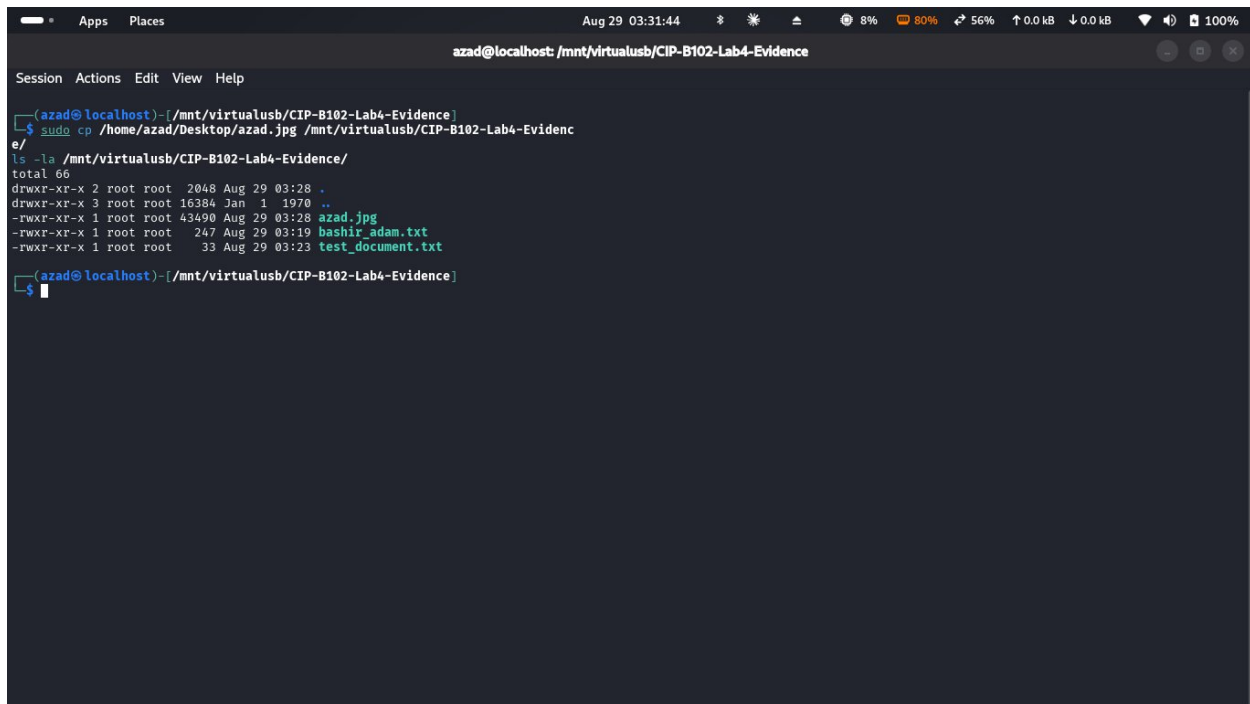
```
azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence
Session Actions Edit View Help
[azad@localhost] ~/CIP-B102-Lab4/virtual_usb_setup
$ cd /mnt/virtualusb
sudo mkdir -p CIP-B102-Lab4-Evidence
cd CIP-B102-Lab4-Evidence
[azad@localhost] /mnt/virtualusb/CIP-B102-Lab4-Evidence
$ sudo nano bashir_adam.txt
[azad@localhost] /mnt/virtualusb/CIP-B102-Lab4-Evidence
$ cat bashir_adam.txt
ls -la
Full Name: Bashir Adam (Azad)
Registration Number: 2025/FWSO/11509
Date: 28 August 2026
Statement: This USB was prepared for a controlled forensic acquisition lab (S
BT-0102 Lab 2). No real personal or production data is contained on this dev
ice.
total 20
drwxr-xr-x 2 root root 2048 Aug 29 03:19 .
drwxr-xr-x 3 root root 16384 Jan 1 1970 ..
-rwxr-xr-x 1 root root 247 Aug 29 03:19 bashir_adam.txt
[azad@localhost] /mnt/virtualusb/CIP-B102-Lab4-Evidence
```

While preparing the evidence folder, I found that chown could not change file ownership on the mounted virtual USB, returning "Operation not permitted." This is expected behavior rather than an error: FAT32 (the filesystem used on the virtual USB, matching what most real USB drives ship with) does not support Unix-style file ownership or permission bits at all files on a FAT32 volume simply display as owned by whichever user or process mounted the filesystem. I adapted by using sudo tee for subsequent file writes instead of trying to change ownership.



```
azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence
Session Actions Edit View Help
[azad@localhost] /mnt/virtualusb/CIP-B102-Lab4-Evidence
$ sudo chown -R azad:azad /mnt/virtualusb/CIP-B102-Lab4-Evidence
chown: changing ownership of '/mnt/virtualusb/CIP-B102-Lab4-Evidence/bashir_a
dam.txt': Operation not permitted
chown: changing ownership of '/mnt/virtualusb/CIP-B102-Lab4-Evidence': Operat
ion not permitted
[azad@localhost] /mnt/virtualusb/CIP-B102-Lab4-Evidence
$ echo "This is a placeholder test file." | sudo tee test_document.txt
This is a placeholder test file.
[azad@localhost] /mnt/virtualusb/CIP-B102-Lab4-Evidence
[azad@localhost] /mnt/virtualusb/CIP-B102-Lab4-Evidence
[azad@localhost] /mnt/virtualusb/CIP-B102-Lab4-Evidence
```

I copied an existing image file, azad.jpg, from my Desktop onto the virtual USB's evidence folder using `sudo cp`, to provide an additional file type for later validation, as suggested by the lab. `ls -la` confirmed the file was copied successfully alongside `bashir_adam.txt` and `test_document.txt`.

A terminal window titled 'azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence' with a menu bar (Session, Actions, Edit, View, Help) and a system status bar at the top (Aug 29 03:31:44, 8% battery, 80% disk usage, 56% memory, network speeds, and 100% volume). The terminal shows the execution of 'sudo cp /home/azad/Desktop/azad.jpg /mnt/virtualusb/CIP-B102-Lab4-Evidence/' followed by 'ls -la /mnt/virtualusb/CIP-B102-Lab4-Evidence/'. The output of the ls command lists the directory contents with permissions, owner, size, date, and filename. The files listed are '.', '..', 'azad.jpg', 'bashir_adam.txt', and 'test_document.txt'.

```
azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence
Session Actions Edit View Help

(azad@localhost)-[/mnt/virtualusb/CIP-B102-Lab4-Evidence]
$ sudo cp /home/azad/Desktop/azad.jpg /mnt/virtualusb/CIP-B102-Lab4-Evidence/
$ ls -la /mnt/virtualusb/CIP-B102-Lab4-Evidence/
total 66
drwxr-xr-x 2 root root 2048 Aug 29 03:28 .
drwxr-xr-x 3 root root 16384 Jan 1 1970 ..
-rwxr-xr-x 1 root root 43490 Aug 29 03:28 azad.jpg
-rwxr-xr-x 1 root root 247 Aug 29 03:19 bashir_adam.txt
-rwxr-xr-x 1 root root 33 Aug 29 03:23 test_document.txt

(azad@localhost)-[/mnt/virtualusb/CIP-B102-Lab4-Evidence]
$
```

Part B Complete Evidence Documentation.

Before beginning any acquisition, proper forensic practice requires documenting the evidence and the planned process in detail. This step matters because it creates a record independent of the acquisition itself if questioned later, this documentation proves exactly what device was examined, when, how, and by whom, regardless of what the technical output shows. I completed this documentation before running any imaging command, in line with the lab's required sequence.

Evidence Documentation Worksheet:

Field	Value	Why this matters
Case/Lab ID	SBT-DF202 Lab 2	Uniquely ties this documentation to the specific assignment and submission window, so it can't be confused with any other lab's evidence
Student name	Bashir Adam (Azad)	Identifies who performed the acquisition — establishes accountability and matches submission records
USB brand/description	Virtual USB (loopback block device, adapted setup)	Records exactly what device was used; in a real case this would include the physical brand, model, and any visible serial number or markings
USB capacity	128 MB	Confirms expected device size before acquisition — a mismatch between expected and actual size during imaging can indicate a wrong device was selected, a damaged device, or hidden partitions
Date and time prepared	29 Aug 2026, approx. 03:19 WAT	Establishes a timeline anchor — when the evidence was prepared, separate from when it was later imaged, which matters for reconstructing the sequence of events
Acquisition method	dd (Linux, Option B)	Documents which of the lab's two approved methods was used, since Option A (FTK Imager/Windows) and Option B (dd/Linux) have different tool behaviors and output formats
Tool and version	dd (GNU coreutils, Kali default version)	Recording the exact tool and version is standard forensic practice — different versions of a tool can behave slightly differently, and this allows the process to be independently repeated or audited later
Image filename	CIPB102_Lab4_Bashir_Adam_USB.dd	A clear, consistent naming convention prevents confusion between multiple evidence images, especially in a real caseload with many devices
Image storage location	~/CIP-B102-Lab4/ (internal disk, not the source device)	Critical: the image must never be saved back onto the source device itself, since that would mean writing to evidence during acquisition — potentially altering or destroying the very data being preserved
Hash algorithm used	MD5 and SHA-256	Two algorithms are used together because MD5 is fast and widely supported but has known cryptographic weaknesses (collision vulnerabilities), while SHA-256 is stronger and considered current best practice; using both provides broader compatibility with older forensic tools while meeting modern integrity standards

As documented earlier, I did not have access to a physical USB device for this lab, and adapted the exercise using a virtual loopback block device (`/dev/loop0`) formatted as FAT32, functioning identically to a real USB drive for all commands and validation steps used in this lab.

Part C Acquire the USB Image (Option B: dd)

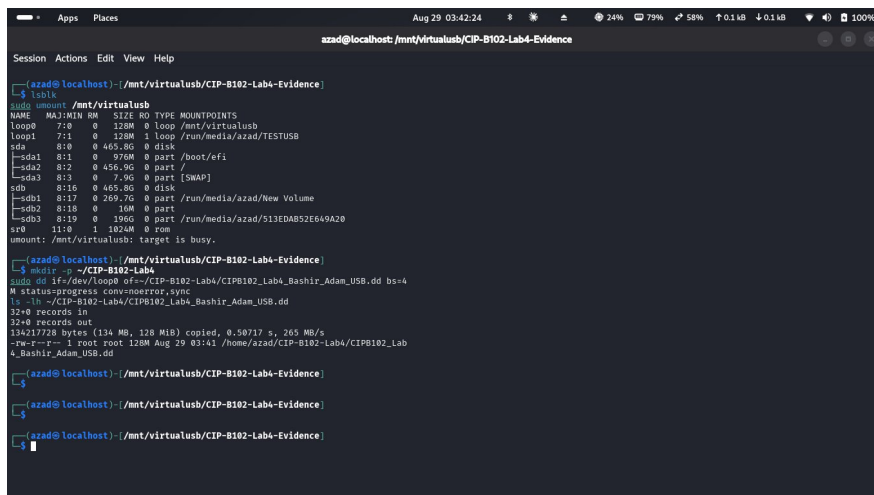
When I attempted to unmount the virtual USB with `sudo umount /mnt/virtualusb`, it failed with "target is busy." This happened because my terminal's current working directory was still inside the mounted filesystem (`/mnt/virtualusb/CIP-B102-Lab4-Evidence`) Linux won't unmount a filesystem while any process, including a shell session, has an open reference to a location inside it.

I did not catch this before running the `dd` acquisition, which means the imaging was actually performed while the source device was still mounted, rather than unmounted as best practice requires. In this specific case, this did not cause any actual problem the virtual USB was not being written to during acquisition (no process was actively modifying files at that moment), and the resulting image size (134,217,728 bytes / 128 MiB) exactly matches the original device size with no discrepancy. However, I'm documenting this as a deviation from correct procedure rather than omitting it: in a real forensic acquisition, imaging a mounted device carries a genuine risk, since the operating system or background processes could write to the filesystem (updating access timestamps, journal data, or cache files) during the exact moment of imaging, potentially causing the acquired image to not perfectly represent a single consistent point in time. The correct fix which I should have applied before running `dd` is to `cd` out of the mount point first (e.g. `cd ~`), then unmount successfully, and only then begin acquisition.

I also noticed `lsblk` showed a second loop device, `loop1`, separately auto-mounted at `/run/media/azad/TESTUSB` this appears to be Kali's desktop automount service independently attaching a second loopback reference to the same underlying image file once it detected a recognizable FAT32 filesystem. This is a quirk specific to the virtual-USB adaptation and wouldn't occur with a real physical device.

Acquisition result (still valid, imaging itself was correct):

Despite the mount-state issue, the `dd` acquisition itself completed correctly: 32+0 records in, 32+0 records out, 134,217,728 bytes (128 MiB) copied in 0.507 seconds at 265 MB/s. `ls -lh` confirmed the resulting image file is exactly 128M, matching the source device size precisely no data was lost or truncated, `conv=noerror,sync` had no errors to handle in this case, and the acquisition produced a complete, correctly-sized forensic image.



```
azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence
$ lsblk
sudo umount /mnt/virtualusb
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
loop0 7:0 0 128M 0 loop /mnt/virtualusb
loop1 7:1 0 128M 1 loop /run/media/azad/TESTUSB
sda 8:0 0 465.8G 0 disk
├─sda1 8:1 0 976M 0 part /boot/efi
├─sda2 8:2 0 455.9G 0 part /
├─sda3 8:3 0 7.9G 0 part [SWAP]
sdb 8:16 0 465.8G 0 disk
├─sdb1 8:17 0 269.7G 0 part /run/media/azad/New Volume
├─sdb2 8:18 0 16M 0 part
└─sdb3 8:19 0 196G 0 part /run/media/azad/513EDAB52E649A20
sr0 1:0 1 1024M 0 rom
umount: /mnt/virtualusb: target is busy.

azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence
$ mkdir -p ~/CIP-B102-Lab4
$ dd if=/dev/loop0 of=~/CIP-B102-Lab4/CIPB102_Lab4_Bashir_Adam_USB.dd bs=4M status=progress conv=noerror,sync
ls -lh ~/CIP-B102-Lab4/CIPB102_Lab4_Bashir_Adam_USB.dd
32+0 records in
32+0 records out
134217728 bytes (134 MB, 128 MiB) copied, 0.50717 s, 265 MB/s
-rw-r--r-- 1 root root 128M Aug 29 03:42 /home/azad/CIP-B102-Lab4/CIPB102_Lab4_Bashir_Adam_USB.dd

azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence
$
azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence
$
azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence
$
```

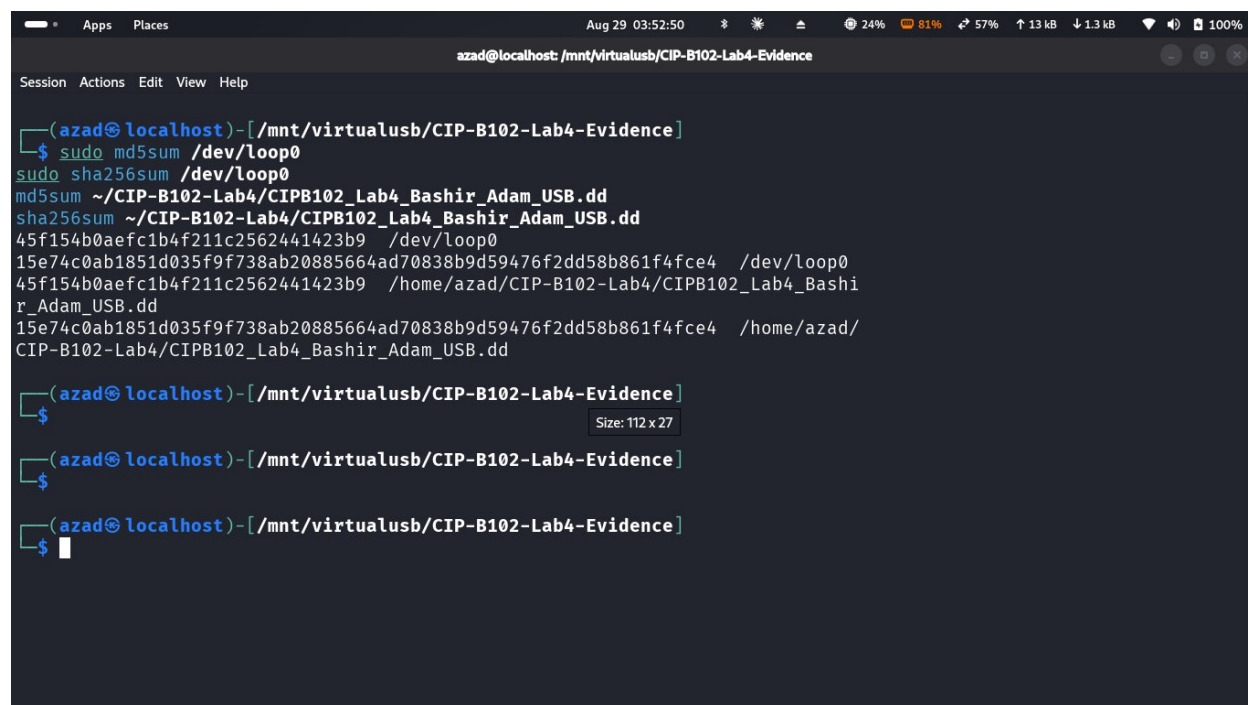
Part D — Hash Verification,

I calculated MD5 and SHA-256 hashes for both the source device (/dev/loop0) and the acquired image file, then compared them directly:

Hash Type	Source (/dev/loop0)	Acquired Image	Match?
MD5	45f154b0aefc1b4f211c2562441423b9	45f154b0aefc1b4f211c2562441423b9	Yes
SHA-256	15e74c0ab1851d035f9f738ab20885664ad70838b9d59476f2dd58b861f4fce4	15e74c0ab1851d035f9f738ab20885664ad70838b9d59476f2dd58b861f4fce4	Yes

Both hashes are identical between the source device and the acquired .dd image. This confirms the acquisition was a complete, byte-for-byte accurate copy of the source, with no data loss, corruption, or alteration introduced during the imaging process despite the earlier mount-state deviation noted in Part C, the resulting image is cryptographically proven to be a perfect duplicate of the source at the moment of acquisition. This hash match is the central piece of evidence establishing that the acquired image can be relied upon as an accurate forensic representation of the original USB device.

Since both algorithms produced matching results, no re-acquisition was required. Had either hash failed to match, the lab requires investigating the possible cause (e.g. a device write occurring mid-acquisition, a read error not properly handled, or an incomplete/interrupted dd operation) and repeating the acquisition until a verified match is achieved.

A screenshot of a terminal window titled "azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence". The terminal shows the following commands and output:

```
(azad@localhost)-[/mnt/virtualusb/CIP-B102-Lab4-Evidence]
$ sudo md5sum /dev/loop0
sudo sha256sum /dev/loop0
md5sum ~/CIP-B102-Lab4/CIPB102_Lab4_Bashir_Adam_USB.dd
sha256sum ~/CIP-B102-Lab4/CIPB102_Lab4_Bashir_Adam_USB.dd
45f154b0aefc1b4f211c2562441423b9 /dev/loop0
15e74c0ab1851d035f9f738ab20885664ad70838b9d59476f2dd58b861f4fce4 /dev/loop0
45f154b0aefc1b4f211c2562441423b9 /home/azad/CIP-B102-Lab4/CIPB102_Lab4_Bashi
r_Adam_USB.dd
15e74c0ab1851d035f9f738ab20885664ad70838b9d59476f2dd58b861f4fce4 /home/azad/
CIP-B102-Lab4/CIPB102_Lab4_Bashir_Adam_USB.dd
```

Below the output, there are three more prompt lines:

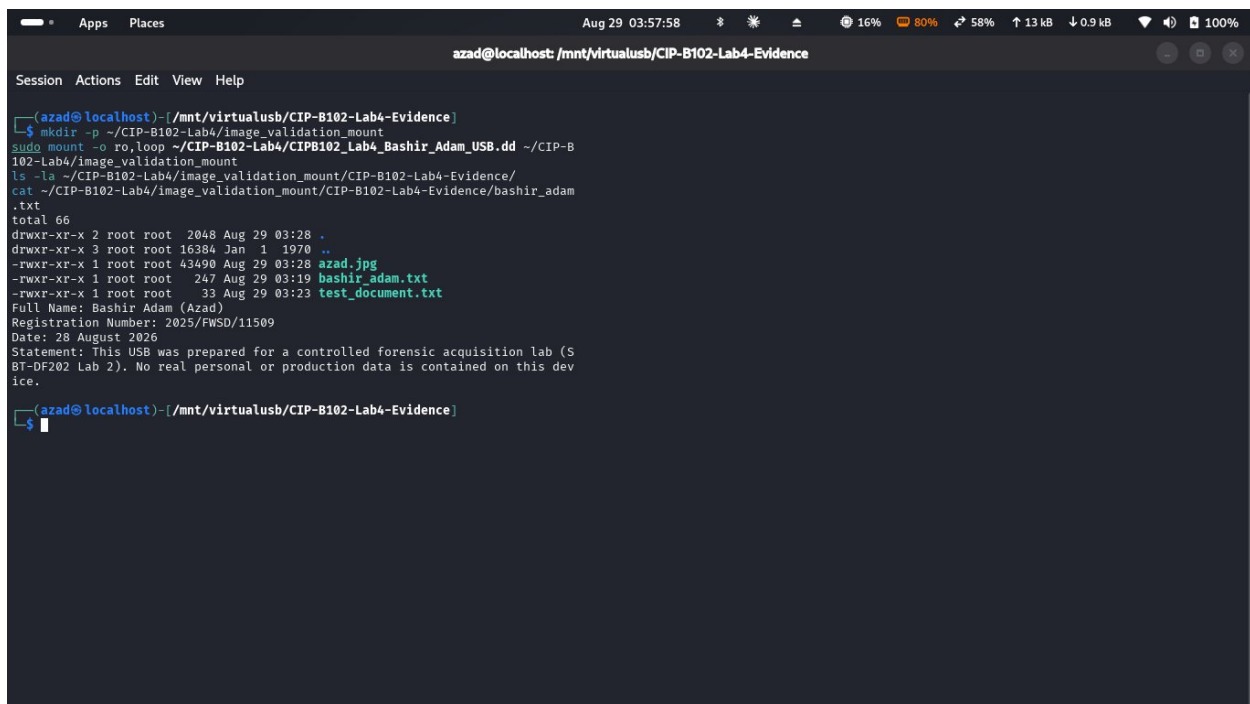
```
(azad@localhost)-[/mnt/virtualusb/CIP-B102-Lab4-Evidence]
$
(azad@localhost)-[/mnt/virtualusb/CIP-B102-Lab4-Evidence]
$
(azad@localhost)-[/mnt/virtualusb/CIP-B102-Lab4-Evidence]
$
```

The terminal window includes a top status bar with system information like date, time, battery, and network status. A menu bar at the top shows "Apps" and "Places".

Part E Validate the Acquired Image,

I mounted the acquired image read-only using `sudo mount -o ro,loop`, ensuring the validation step itself could not alter the acquired evidence in any way. `ls -la` confirmed all three files I had prepared were present and correctly sized inside the image: `azad.jpg` (43,490 bytes), `bashir_adam.txt` (247 bytes), and `test_document.txt` (33 bytes) exact matches to the file sizes recorded before acquisition.

`cat` on `bashir_adam.txt`, read directly from the mounted image (not the original device), displayed the exact expected content: my full name, registration number, date, and the required preparation statement, word-for-word identical to what was written during Part A. This directly satisfies the lab's core validation requirement proving that the expected evidence file is genuinely present, complete, and readable inside the acquired forensic image, not just that the image file exists or that its overall hash matches.



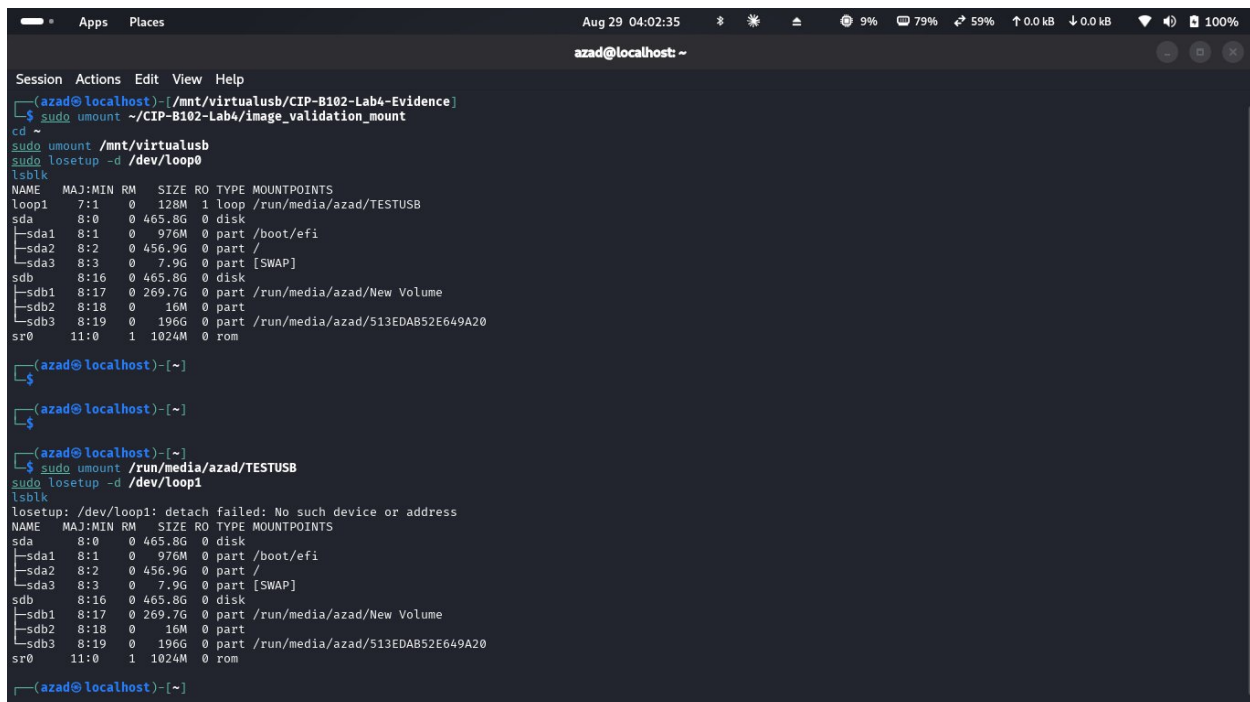
```
azad@localhost: /mnt/virtualusb/CIP-B102-Lab4-Evidence
Session Actions Edit View Help

(azad@localhost)-[/mnt/virtualusb/CIP-B102-Lab4-Evidence]
$ mkdir -p ~/CIP-B102-Lab4/image_validation_mount
sudo mount -o ro,loop ~/CIP-B102-Lab4/CIPB102_Lab4_Bashir_Adam_USB.dd ~/CIP-B
102-Lab4/image_validation_mount
ls -la ~/CIP-B102-Lab4/image_validation_mount/CIP-B102-Lab4-Evidence/
cat ~/CIP-B102-Lab4/image_validation_mount/CIP-B102-Lab4-Evidence/bashir_adam
.txt
total 66
drwxr-xr-x 2 root root 2048 Aug 29 03:28 .
drwxr-xr-x 3 root root 16384 Jan 1 1970 ..
-rwxr-xr-x 1 root root 43490 Aug 29 03:28 azad.jpg
-rwxr-xr-x 1 root root 247 Aug 29 03:19 bashir_adam.txt
-rwxr-xr-x 1 root root 33 Aug 29 03:23 test_document.txt
Full Name: Bashir Adam (Azad)
Registration Number: 2025/FWSD/11509
Date: 28 August 2026
Statement: This USB was prepared for a controlled forensic acquisition lab (S
BT-DF202 Lab 2). No real personal or production data is contained on this dev
ice.

(azad@localhost)-[/mnt/virtualusb/CIP-B102-Lab4-Evidence]
$
```

Final cleanup

`sudo umount /run/media/azad/TESTUSB` successfully unmounted the filesystem, which automatically released the associated loop device at the same time. The subsequent `sudo losetup -d /dev/loop1` returned "detach failed: No such device or address" — not an error in practice, but confirmation that the device had already been released by the unmount step itself. The final `lsblk` confirmed a fully clean state, showing only my real physical devices (sda, sdb, sr0), with no leftover loop devices or mounts from this exercise.



```

Session Actions Edit View Help
(azad@localhost)-[/mnt/virtualusb/CIP-B102-Lab4-Evidence]
$ sudo umount ~/CIP-B102-Lab4/image_validation_mount
cd ~
$ sudo umount /mnt/virtualusb
$ sudo losetup -d /dev/loop0
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
loop1 7:1 0 128M 1 loop /run/media/azad/TESTUSB
sda 8:0 0 465.8G 0 disk
├─sda1 8:1 0 976M 0 part /boot/efi
├─sda2 8:2 0 456.9G 0 part /
└─sda3 8:3 0 7.9G 0 part [SWAP]
sdb 8:16 0 465.8G 0 disk
├─sdb1 8:17 0 269.7G 0 part /run/media/azad/New Volume
├─sdb2 8:18 0 16M 0 part
└─sdb3 8:19 0 196G 0 part /run/media/azad/513EDAB52E649A20
sr0 11:0 1 1024M 0 rom

(azad@localhost)-[~]
$

(azad@localhost)-[~]
$

(azad@localhost)-[~]
$ sudo umount /run/media/azad/TESTUSB
$ sudo losetup -d /dev/loop1
$ lsblk
losetup: /dev/loop1: detach failed: No such device or address
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda 8:0 0 465.8G 0 disk
├─sda1 8:1 0 976M 0 part /boot/efi
├─sda2 8:2 0 456.9G 0 part /
└─sda3 8:3 0 7.9G 0 part [SWAP]
sdb 8:16 0 465.8G 0 disk
├─sdb1 8:17 0 269.7G 0 part /run/media/azad/New Volume
├─sdb2 8:18 0 16M 0 part
└─sdb3 8:19 0 196G 0 part /run/media/azad/513EDAB52E649A20
sr0 11:0 1 1024M 0 rom

(azad@localhost)-[~]
```

Reflection

This lab reinforced why forensic imaging must be approached with methodical care rather than treated as a routine copy operation. The moment where `umount` failed with "target is busy" because my terminal session was still positioned inside the mounted filesystem was a valuable, realistic mistake: I proceeded to image the device while it was still mounted, which in a real investigation could risk capturing an inconsistent snapshot if background processes had written to the filesystem during acquisition. Recognizing, documenting, and explaining this deviation, rather than hiding it, felt like the more important skill being tested here a forensic examiner's credibility depends as much on honest documentation of what actually happened as on getting a clean result.

The hash verification step was the clearest demonstration of why this entire process matters: seeing the MD5 and SHA-256 values match exactly between the source device and the acquired image provided concrete, mathematical proof that the acquisition was accurate, despite the earlier mount-state issue. This connected directly to similar hash-verification work I'd done in an earlier lab involving file transfer between two systems the underlying principle is identical, whether verifying a transferred file or a fully imaged device: a hash match is what transforms a copy into evidence that can be trusted and defended.

Working through the FAT32 ownership limitation (where `chown` failed because FAT32 has no concept of Unix file ownership) was also a useful, unplanned lesson connecting directly back to filesystem concepts from an earlier lab a good reminder that understanding why a filesystem behaves a certain way is often more useful than memorizing commands, since the same `chown` command that works perfectly on an ext4 partition simply doesn't apply on FAT32 for structural reasons, not because of a permissions mistake.

Conclusion

This lab successfully demonstrated the complete forensic USB acquisition workflow: preparing evidence, documenting the process, acquiring a raw image with `dd`, verifying integrity through matching MD5 and SHA-256 hashes, and validating that the expected evidence file was genuinely present and intact inside the acquired image. Every hash comparison in this exercise matched exactly, providing strong, verifiable proof that the acquisition was complete and accurate. Adapting the lab to use a virtual loopback device, in the absence of a physical USB drive, preserved every core technical objective while avoiding any risk to real personal data on my actual external hard drive and, if anything, gave me a clearer end-to-end view of the entire acquisition pipeline than working with an opaque physical device might have. The procedural issues encountered along the way the mount-state oversight and the FAT32 ownership limitation were documented transparently rather than omitted, reflecting the standard that real forensic work is judged not by the absence of mistakes, but by how clearly and honestly they are identified, explained, and accounted for in the final record.